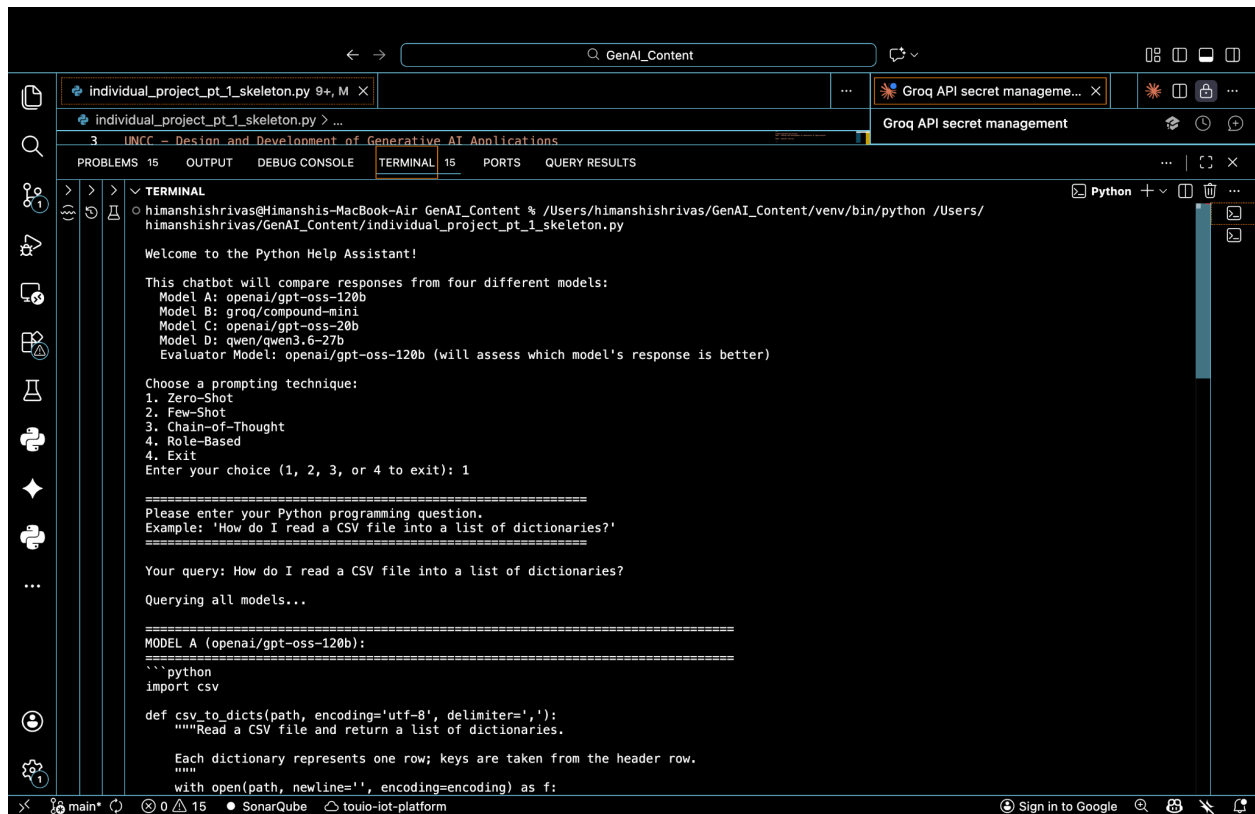


Screenshots-



```
GenAI_Content

individual_project_pt_1_skeleton.py 9+, M X
individual_project_pt_1_skeleton.py > ...
UNCC - Design and Development of Generative AI Applications
PROBLEMS 15 OUTPUT DEBUG CONSOLE TERMINAL 15 PORTS QUERY RESULTS
Python + v ...

himanshishrivass@Himanshis-MacBook-Air GenAI_Content % /Users/himanshishrivass/GenAI_Content/venv/bin/python /Users/himanshishrivass/GenAI_Content/individual_project_pt_1_skeleton.py

Welcome to the Python Help Assistant!

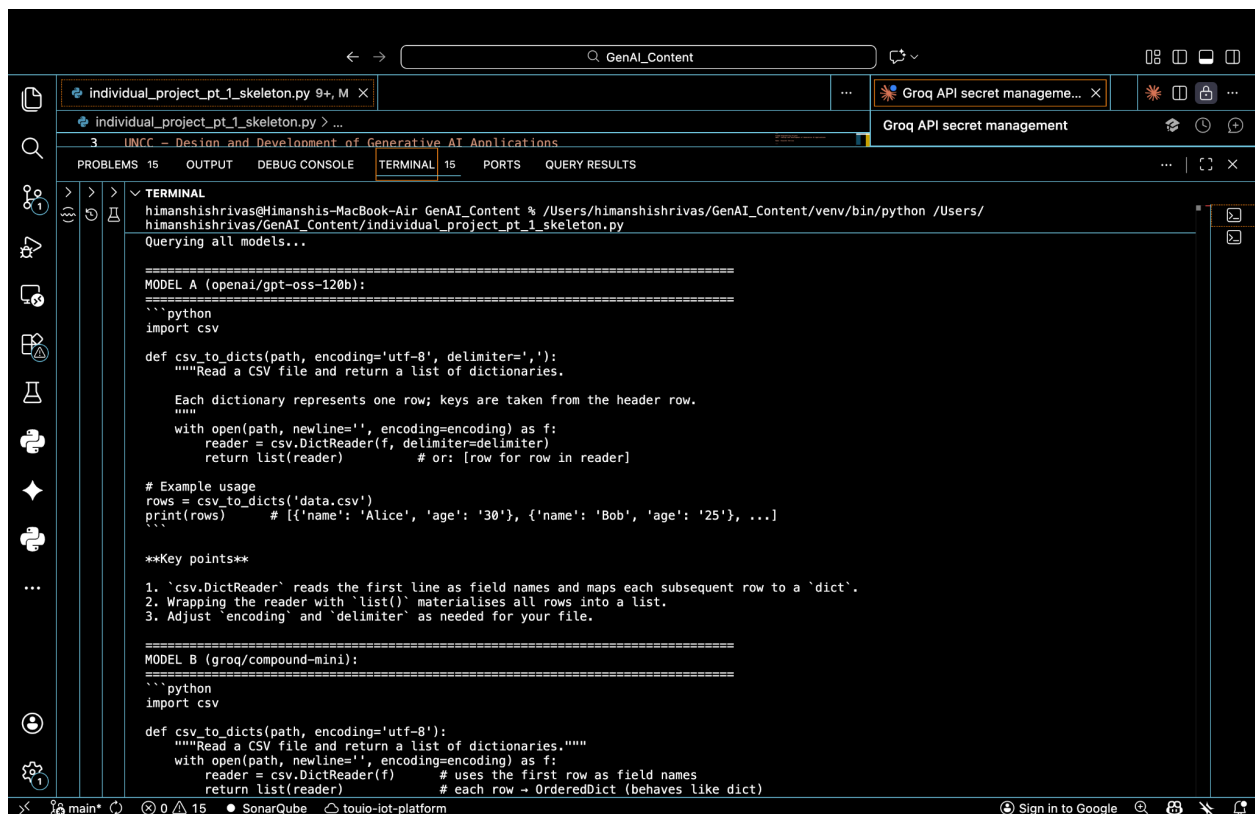
This chatbot will compare responses from four different models:
Model A: openai/gpt-oss-120b
Model B: groq/compound-mini
Model C: openai/gpt-oss-20b
Model D: qwen/qwen3.6-27b
Evaluator Model: openai/gpt-oss-120b (will assess which model's response is better)

Choose a prompting technique:
1. Zero-Shot
2. Few-Shot
3. Chain-of-Thought
4. Role-Based
5. Exit
Enter your choice (1, 2, 3, or 4 to exit): 1

Please enter your Python programming question.
Example: 'How do I read a CSV file into a list of dictionaries?'

Your query: How do I read a CSV file into a list of dictionaries?

Querying all models...
```



```
GenAI_Content

individual_project_pt_1_skeleton.py 9+, M X
individual_project_pt_1_skeleton.py > ...
UNCC - Design and Development of Generative AI Applications
PROBLEMS 15 OUTPUT DEBUG CONSOLE TERMINAL 15 PORTS QUERY RESULTS
Python + v ...

himanshishrivass@Himanshis-MacBook-Air GenAI_Content % /Users/himanshishrivass/GenAI_Content/venv/bin/python /Users/himanshishrivass/GenAI_Content/individual_project_pt_1_skeleton.py

Querying all models...

MODEL A (openai/gpt-oss-120b):
=====
```python
import csv

def csv_to_dicts(path, encoding='utf-8', delimiter=','):
 """Read a CSV file and return a list of dictionaries.

 Each dictionary represents one row; keys are taken from the header row.
 """
 with open(path, newline='', encoding=encoding) as f:
 reader = csv.DictReader(f, delimiter=delimiter)
 return list(reader)

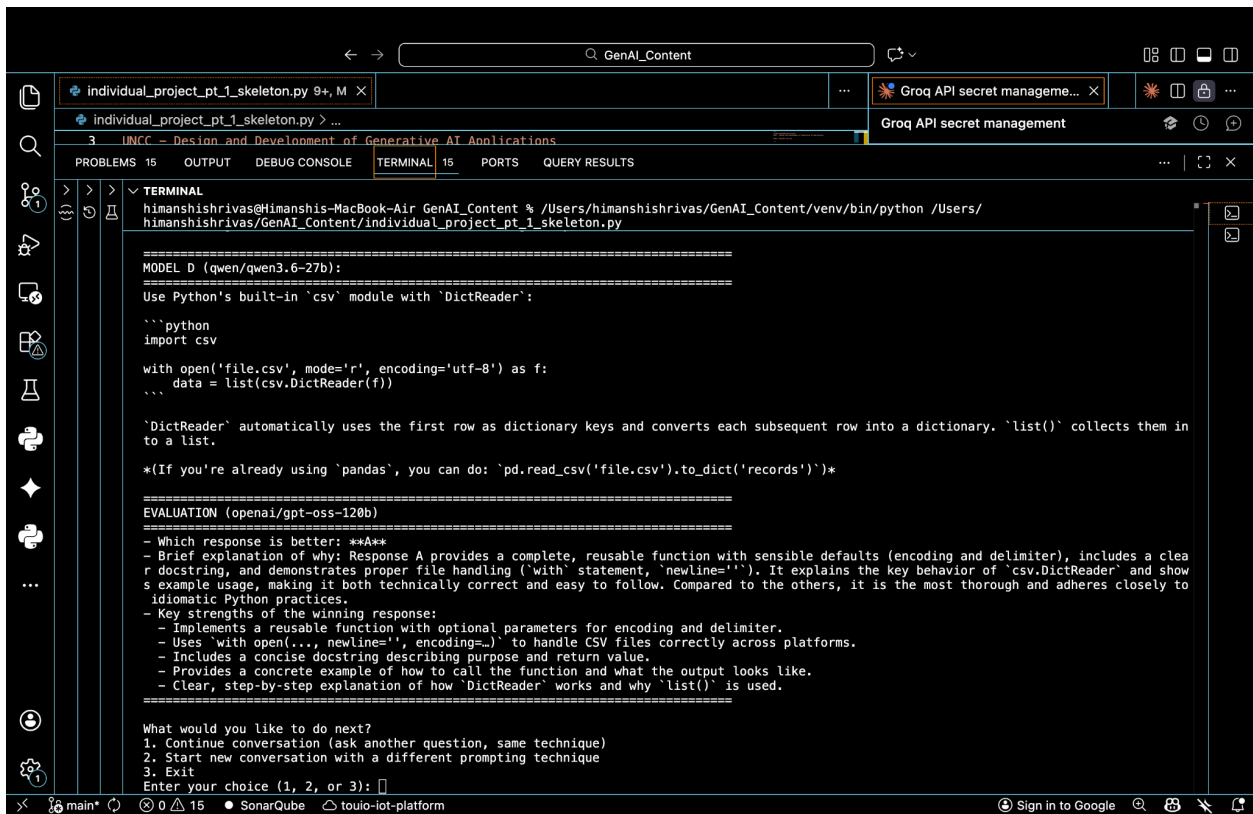
Example usage
rows = csv_to_dicts('data.csv')
print(rows) # [{'name': 'Alice', 'age': '30'}, {'name': 'Bob', 'age': '25'}, ...]

Key points

1. 'csv.DictReader' reads the first line as field names and maps each subsequent row to a 'dict'.
2. Wrapping the reader with 'list()' materialises all rows into a list.
3. Adjust 'encoding' and 'delimiter' as needed for your file.

=====
MODEL B (groq/compound-mini):
=====
```python
import csv

def csv_to_dicts(path, encoding='utf-8'):
    """Read a CSV file and return a list of dictionaries."""
    with open(path, newline='', encoding=encoding) as f:
        reader = csv.DictReader(f)  # uses the first row as field names
        return list(reader)  # each row -> OrderedDict (behaves like dict)
```



Input/Output (Terminal)-

s that the result is a standard dict (Python 3.7+).

What would you like to do next?

1. Continue conversation (ask another question, same technique)
2. Start new conversation with a different prompting technique
3. Exit

Enter your choice (1, 2, or 3): 2

Choose a prompting technique:

1. Zero-Shot
2. Few-Shot
3. Chain-of-Thought
4. Role-Based
5. Exit

Enter your choice (1, 2, 3, 4, or 5 to exit): 2

Please enter your Python programming question.

Example: 'How do I read a CSV file into a list of dictionaries?'

Your query: How do I read a CSV file into a list of dictionaries?

Querying all models...

MODEL A (openai/gpt-oss-120b):

Use the built-in `csv.DictReader` module's `DictReader`. It reads each row as a dictionary whose keys come from the header line, and you can collect those rows in a list.

```

python
import csv

csv_path = "data.csv"      # path to your CSV file
with open(csv_path, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f) # creates an iterator of dicts
    rows = list(reader)       # list of dictionaries

# Example: print the first row
print(rows[0])

```

=====

MODEL B (groq/compound-mini):

=====

A: Use the built-in `csv.DictReader`, which reads each row into a dictionary keyed by the column headers.

```

python
import csv

csv_path = "data.csv"      # path to your CSV file
rows = []                  # will hold the list of dictionaries

with open(csv_path, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f) # reads header row automatically
    for row in reader:
        rows.append(dict(row)) # each `row` is an OrderedDict; convert to plain dict if desired

# rows now contains a list like:
# [{ 'name': 'Alice', 'age': '30' }, { 'name': 'Bob', 'age': '25' }, ...]
print(rows)

```

=====

MODEL C (openai/gpt-oss-20b):

=====

Q: How do I read a CSV file into a list of dictionaries?

A: Use the `csv.DictReader` class—it reads each row as a `dict` keyed by the header names.

```

python
import csv

with open('data.csv', newline='') as f:
    reader = csv.DictReader(f)
    rows = list(reader) # list of dicts

```

```

# Example output
# [{ 'name': 'Alice', 'age': '30' }, { 'name': 'Bob', 'age': '25' }]

```

=====

MODEL D (qwen/qwen3.6-27b):

=====

Use the `csv.DictReader` class, which automatically uses the first row as dictionary keys.

```

python
import csv

with open('data.csv', 'r') as file:
    data = list(csv.DictReader(file))

```

=====

EVALUATION (openai/gpt-oss-120b)

=====

- Which response is better: ****A****

- Brief explanation of why: Response A correctly uses `csv.DictReader` to produce a list of dictionaries, includes the recommended `newline=""` and explicit UTF-8 encoding for robust cross-platform handling, and presents the code in a concise, easy-to-read block with a simple demonstration. It balances completeness with brevity better than the others.

- Key strengths of the winning response:

- ****Correctness**** Proper use of `csv.DictReader` and `list(reader)` yields the desired list of dicts.

- ****Clarity**** Clear comments, straightforward variable names, and a concrete example (`print(rows[0])`).

- ****Best practices**** Uses `newline=""` and `encoding="utf-8"` when opening the file, which avoids common CSV parsing pitfalls on Windows and ensures Unicode handling.

- ****Conciseness**** Provides the solution without unnecessary loops or conversions, keeping the code idiomatic and readable.

=====

What would you like to do next?

1. Continue conversation (ask another question, same technique)
2. Start new conversation with a different prompting technique
3. Exit

Enter your choice (1, 2, or 3): 2

Choose a prompting technique:

1. Zero-Shot
2. Few-Shot
3. Chain-of-Thought
4. Role-Based
5. Exit

Enter your choice (1, 2, 3, 4, or 5 to exit): 3

```
=====
Please enter your Python programming question.
Example: 'How do I read a CSV file into a list of dictionaries?'
=====
```

Your query: How do I read a CSV file into a list of dictionaries?

Querying all models...

```
=====
MODEL A (openai/gpt-oss-120b):
=====
**1. What the question is really asking**
```

You have a CSV (comma-separated values) file that looks something like a table:

```
...
name,age,city
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
...
```

You want to load that data into Python **as a list where each row is represented by a dictionary**.

The dictionary keys should be the column headers ('name', 'age', 'city') and the values should be the corresponding cell contents for each row.

****2. How to solve it – step-by-step logic****

```
| Step | What to do | Why |
|-----|-----|-----|
| **a.** Import the 'csv' module | `import csv` | The standard library provides tools for parsing CSV files safely. |
| **b.** Open the file | `with open('myfile.csv', newline="", encoding='utf-8') as f:` | `newline=""` prevents Python from translating line endings, which `csv` expects. `encoding` ensures you read the file correctly. |
| **c.** Create a 'DictReader' | `reader = csv.DictReader(f)` | 'DictReader' reads each row and automatically builds a 'dict' using the first line of the file as the keys. |
| **d.** Iterate over the reader and collect rows | `data = list(reader)` | The 'reader' is an iterator that yields one dictionary per row. Converting it to a list gives you the desired structure. |
| **e.** (Optional) Convert values to proper types | e.g., `row['age'] = int(row['age'])` | CSV stores everything as strings; you may want numbers, dates, etc. |
```

****3. Complete answer with a runnable code example****

Below is a self-contained example. It:

1. Creates a small CSV file ('people.csv') for demonstration.
2. Reads that file into a list of dictionaries.
3. Shows how you could post-process a column (convert 'age' to 'int').

```
```python

1. (Optional) Create a sample CSV file to work with

sample_csv = """name,age,city
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
"""
```

```
with open('people.csv', 'w', newline="", encoding='utf-8') as f:
 f.write(sample_csv)
```

```

```

# 2. Read the CSV into a list of dictionaries

# -----

import csv

def read\_csv\_as\_dicts(filepath):

"""

Reads a CSV file and returns a list where each element is a dict mapping column names to cell values.

"""

with open(filepath, newline="", encoding='utf-8') as f:

# csv.DictReader automatically uses the first row as field names

reader = csv.DictReader(f)

# Convert the iterator to a list

rows = list(reader)

# Optional: convert data types (example: make 'age' an int)

for row in rows:

# All values are strings by default; convert what you need

if 'age' in row and row['age'].isdigit():

row['age'] = int(row['age'])

return rows

# -----

# 3. Use the function and inspect the result

# -----

people = read\_csv\_as\_dicts('people.csv')

print(people)

...

**\*\*Output\*\***

```python

[

{'name': 'Alice', 'age': 30, 'city': 'New York'},

{'name': 'Bob', 'age': 25, 'city': 'Los Angeles'},

{'name': 'Charlie', 'age': 35, 'city': 'Chicago'}]

]

...

****Key points to remember****

* `csv.DictReader` does the heavy lifting: it reads the header row once and builds a dict for each subsequent line.

* The result is an iterator; wrapping it with `list()` gives you the full list at once.

* All values come in as strings; cast them to `int`, `float`, `datetime`, etc., as needed.

* If your CSV uses a different delimiter (e.g., tabs or semicolons), pass `delimiter='\t'` or `delimiter=';'` to `DictReader`.

That's all you need to turn a CSV file into a list of dictionaries in pure Python!

=====

MODEL B (groq/compound-mini):

=====

****1. What the question is really asking****

You have a CSV (comma-separated values) file, for example:

...

name,age,city

Alice,30,New York

Bob,25,Los Angeles

...

You want to load that file so that each row becomes a Python `dict` where the keys are the column headers and the values are the cell contents. All those dictionaries should be collected in a single `list`:

```python

[

{"name": "Alice", "age": "30", "city": "New York"},

{"name": "Bob", "age": "25", "city": "Los Angeles"},

]

...

**\*\*2. Reasoning through the solution\*\***

1. **\*\*Open the file\*\*** – Use `open()` (or `with open(...)` as f: for automatic cleanup).

2. **\*\*Parse CSV rows\*\*** – The standard library module `csv` knows how to split lines correctly, handling quoted fields, different delimiters, etc.

3. **\*\*Create dictionaries automatically\*\*** – `csv.DictReader` reads the first line as the header row and returns each subsequent row as an `OrderedDict` (or plain `dict` in recent Python).
4. **\*\*Collect rows\*\*** – Iterate over the `DictReader` and append each row to a list.
5. **\*\*Optional Type conversion\*\*** – CSV values are read as strings. If you need numbers, convert them after loading.

**\*\*3. Complete answer with code example\*\***

```

'''python
import csv
from pathlib import Path

def csv_to_dicts(csv_path: str | Path) -> list[dict]:
 """
 Read a CSV file and return a list of dictionaries.

 Parameters

 csv_path : str or pathlib.Path
 Path to the CSV file.

 Returns

 list[dict]
 Each dict corresponds to a row; keys are column headers.
 """
 rows: list[dict] = []
 # Using `with` ensures the file is closed automatically
 with open(csv_path, newline="", encoding='utf-8') as f:
 # DictReader reads the first line as field names
 reader = csv.DictReader(f)
 for row in reader:
 # `row` is already a dict; you can copy it if you want a plain dict
 rows.append(dict(row))
 return rows

Example usage

if __name__ == "__main__":
 # Suppose we have a file called "people.csv" with:
 # name,age,city
 # Alice,30,New York
 # Bob,25,Los Angeles
 csv_file = "people.csv"
 data = csv_to_dicts(csv_file)

 # Print the resulting list
 for i, record in enumerate(data, start=1):
 print(f"Row {i}: {record}")

 # Optional: convert numeric fields
 for record in data:
 record["age"] = int(record["age"]) # now age is an int
 print("\nAfter converting ages to int:", data)
'''

```

**\*\*Explanation of the code\*\***

| Step | What happens                                         | Why it matters                                                                                                                                             |
|------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | `import csv`                                         | Loads the CSV parsing utilities.   No external dependencies needed.                                                                                        |
| 2    | `with open(..., newline="", encoding='utf-8') as f:` | Opens the file safely. `newline=""` lets `csv` handle newlines correctly across platforms.   Prevents file-handle leaks and ensures proper line handling.  |
| 3    | `csv.DictReader(f)`                                  | Creates an iterator that yields each row as a dictionary, using the first CSV line as keys.   You get the desired "list of dicts" structure automatically. |
| 4    | `rows.append(dict(row))`                             | Adds a *plain* `dict` to the list (instead of the `OrderedDict` that older Python versions return).   Guarantees a standard mutable dict.                  |
| 5    | Optional conversion (`int(record["age"])`)           | Turns string numbers into actual integers.   Makes downstream numeric operations easier.                                                                   |

**\*\*Alternative with pandas (if you're already using that library)\*\***

```

'''python
import pandas as pd

df = pd.read_csv("people.csv")
'''

```

```
list_of_dicts = df.to_dict(orient="records")
print(list_of_dicts)
...
```

`pandas` does the same thing in one line, but it adds a heavy dependency. For simple scripts, the `csv` module shown above is usually preferred.

```
=====
MODEL C (openai/gpt-oss-20b):
=====
```

```
What the question is really asking
```

You have a CSV (Comma-Separated Values) file that looks something like this:

```
```csv
name,age,city
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
```
```

You want to load that file into Python so that you end up with a *\*list\** where each element is a *\*dictionary\**:

```
```python
[
    {"name": "Alice", "age": "30", "city": "New York"},
    {"name": "Bob", "age": "25", "city": "Los Angeles"},
    {"name": "Charlie", "age": "35", "city": "Chicago"}
]
```
```

So the task is:

1. Open the file safely.
2. Read each row.
3. Turn each row into a dictionary (`'header' → 'value'`).
4. Collect all those dictionaries into a list.

**\*\*How to do it – step-by-step logic\*\***

1. **\*\*Import the right module\*\***  
The standard library has a `csv` module that knows how to parse CSV files.
2. **\*\*Open the file with a context manager\*\*** (`'with open(...)' as f:`) so it closes automatically.
3. **\*\*Create a `csv.DictReader`\*\*** – this object reads the first row as the header and yields each subsequent row as a dictionary.
4. **\*\*Iterate over the `DictReader`\*\*** and append each dictionary to a list, or just convert the whole iterator to a list in one line.
5. (Optional) **\*\*Handle encoding or delimiters\*\*** if your file isn't plain ASCII or comma-delimited.

**\*\*Complete, minimal example\*\***

```
```python
import csv

def csv_to_dict_list(path, *, encoding='utf-8', delimiter=','):
    """
    Read a CSV file and return a list of dictionaries.

    Parameters
    -----
    path : str
        Path to the CSV file.
    encoding : str, optional
        File encoding (default: 'utf-8').
    delimiter : str, optional
        Character that separates fields (default: ',').

    Returns
    -----
    list[dict]
        Each dict maps column names to values from one row.
    """
    with open(path, mode='r', encoding=encoding, newline='') as f:
        reader = csv.DictReader(f, delimiter=delimiter)
        # Convert the iterator to a list
        return list(reader)
```
```

```
Example usage
if __name__ == "__main__":
 csv_file = 'people.csv' # replace with your file path
 people = csv_to_dict_list(csv_file)
 for person in people:
 print(person)
...
```

**\*\*Explanation of the key parts\*\***

- `newline=""` tells Python not to translate newline characters – the `csv` module handles them itself.
- `DictReader` automatically uses the first row as keys, so you don't need to provide a header list manually.
- `list(reader)` consumes the entire iterator and produces the list of dictionaries you asked for.

**\*\*What if you want to keep values as the right types?\*\***

`csv.DictReader` returns all values as strings. If you need integers, floats, or dates you'll have to convert them yourself after reading:

```
```python
people = csv_to_dict_list(csv_file)
for p in people:
    p['age'] = int(p['age']) # convert age to int
...
```
```

**\*\*Alternative: Using pandas (quick and powerful)\*\***

If you're already using `pandas`, it's even simpler:

```
```python
import pandas as pd

df = pd.read_csv(csv_file)
list_of_dicts = df.to_dict(orient='records')
...
```
```

But for most simple scripts the built-in `csv` module is lightweight and perfectly adequate.

```
=====
MODEL D (qwen/qwen3.6-27b):
=====
Error querying the LLM: Error code: 429 - {'error': {'message': 'Rate limit reached for model `qwen/qwen3.6-27b` in organization `org_01jqcgtmc1e8ktt3wma9n5qf7q` service tier `on_demand` on output tokens per minute (OTPM): Limit 1000, Used 478, Requested 578. Please try again in 3.36s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}

=====
EVALUATION (openai/gpt-oss-120b)
=====
- Which response is better: **B**
- Brief explanation of why: All four answers are technically correct, but response B combines accurate code with clear, well-structured explanations and follows modern Python best-practice conventions (type hints, pathlib, explicit conversion to plain dict, comprehensive docstring). Its step-by-step reasoning and concise tables make it easy to follow, while still covering optional type conversion and an alternative pandas solution.

- Key strengths of the winning response:
 Correctness: Uses `csv.DictReader` correctly, handles newline/encoding, returns the desired list of dictionaries.
 Clarity: Organized sections, bullet-point reasoning, and a table that maps each step to its purpose.
 Idiomatic style: Includes type hints (`str | Path`, `list[dict]`), uses `Pathlib` for path handling, converts `OrderedDict` to a plain `dict`, and provides a clean, reusable function with a docstring.
 Completeness: Shows example usage, optional numeric conversion, and even a brief pandas alternative, giving the reader multiple practical options.
=====
```

What would you like to do next?

1. Continue conversation (ask another question, same technique)
  2. Start new conversation with a different prompting technique
  3. Exit
- Enter your choice (1, 2, or 3): 2

Choose a prompting technique:

1. Zero-Shot
2. Few-Shot
3. Chain-of-Thought
4. Role-Based
5. Exit

Enter your choice (1, 2, 3, 4, or 5 to exit): 4

```
=====
Please enter your Python programming question.
Example: 'How do I read a CSV file into a list of dictionaries?'
=====
```

Your query: How do I read a CSV file into a list of dictionaries?

Querying all models...

```
=====
MODEL A (openai/gpt-oss-120b):
=====
Reading a CSV into a *list of dictionaries* – a mentor-style walk-through
```

Below is a step-by-step guide you can use when reviewing a junior's code that needs to turn a CSV file into something like

```
```python
[
    {"name": "Alice", "age": "30", "city": "NY"},
    {"name": "Bob", "age": "25", "city": "LA"},
    ...
]
```
```

### ### 1. Why `csv.DictReader` is the right tool

- \* The **standard library** `csv` module already knows how to split rows, handle quoted fields, different delimiters, etc.
- \* `DictReader` **automatically maps** each column header to a key in a dict for every row – no manual `'zip'` gymnastics required.
- \* It returns an **iterator**, so you can stream large files without loading everything into memory at once (you can still collect them into a list if you need random access).

### ### 2. Core pattern (the “canonical” way)

```
```python
from pathlib import Path
import csv
from typing import List, Dict
```

```
def read_csv_to_dicts(
    file_path: Path,
    *,
    delimiter: str = ",",
    encoding: str = "utf-8",
    skip_empty_lines: bool = True,
) -> List[Dict[str, str]]:
```

Read a CSV file and return a list of dictionaries, one per row.

Parameters

file_path:
Path object (or anything accepted by Path) pointing to the CSV file.

delimiter:
Character that separates fields. Defaults to a comma.

encoding:
Text encoding of the file. Defaults to UTF-8.

skip_empty_lines:
If True, rows that are completely empty are ignored.

Returns

`List[Dict[str, str]]`
Each dict maps column header → cell value (all values are strings).

```
"""
# ① Open the file with the correct newline handling.
# Using `newline=""` tells the csv module to manage line endings itself.
with file_path.open(newline="", encoding=encoding) as f:
    # ② Create the DictReader – it reads the first line as the header row.
    reader = csv.DictReader(f, delimiter=delimiter)

    # ③ Optionally filter out rows that are entirely empty.
    rows = (
```

```

        row for row in reader
        if not (skip_empty_lines and all(v == "" for v in row.values()))
    )

```

```

#4 Materialise the iterator into a list (if you really need a list).
return list(rows)
...

```

****How it works****

Step	What happens	Why it matters
`open(..., newline="")`	Prevents Python from translating `\\n\\n` → `\\n` before the `csv` parser sees the data.	The CSV parser expects the raw line endings; otherwise you can get spurious empty rows on Windows.
`csv.DictReader(f, delimiter=...)`	Reads the "first" line as the header and yields a `dict` for each subsequent line.	Guarantees column names stay in sync with data – no manual `zip`.
`list(rows)`	Pulls the iterator into a concrete list.	If the file is huge you might want to return the iterator itself; the function is flexible enough to be changed later.
`skip\_empty\_lines` guard	Removes rows that are completely blank.	CSV files sometimes have stray newline characters at the end; without this you'd get a dict of empty strings.

3. Common pitfalls (and how to avoid them)

Pitfall	Symptom	Fix / Best practice
****Forgot `newline=""`**	Extra empty rows appear on Windows, or fields with embedded newlines get broken.	Always open with `newline=""` when using `csv`.
****Wrong encoding****	`UnicodeDecodeError` or garbled characters.	Verify the file's encoding (`utf-8`, `utf-8-sig`, `latin-1`, ...) and pass it explicitly.
****Header row missing or duplicated****	`DictReader` uses the first row as keys; you get unexpected keys or overwritten values.	Ensure the CSV has a proper header, or pass `fieldnames=` manually if you want to supply your own list.
****Trailing delimiter**** (e.g., `a,b,c,`)	Last column becomes an empty string key (`""`).	Clean the source file or set `skipinitialspace=True` and/or strip whitespace after reading.
****Mixed data types****	All values are strings, but you later treat them as ints/floats and get `ValueError`.	Convert types ****after**** reading (e.g., `int(row["age"])`). Keep CSV parsing pure – don't embed conversion logic inside the reading function.
****Large files****	`list(reader)` loads everything into RAM → `MemoryError`.	Return the iterator (`return reader`) or use a generator function (`yield from reader`).
****Blank lines inside the file****	You get rows like `{ 'col1': "", 'col2': "" }`.	Use `skip\_empty\_lines=True` (as in the example) or filter with `if any(row.values())`.
****Non-standard delimiter**** (tabs, semicolons)	Data appears in a single column.	Pass the correct `delimiter` argument (`delimiter="\\t"` for TSV, etc.).
****Unexpected quoting**** (e.g., fields contain the delimiter)	Fields get split incorrectly.	The `csv` module handles quoting automatically; just don't disable it unless you really know the format.

4. Quick "one-liner" for a tiny script

If you're writing a throw-away script and you're sure the file is small and well-behaved, you can compress the above to:

```

'''python
import csv
from pathlib import Path

data = list(csv.DictReader(Path("people.csv").open(newline="", encoding="utf-8")))
print(data)
'''

```

But remember the full-featured version for production code – it's more robust and self-documenting.

5. How to test it (a tiny sanity check)

```

'''python
import io
import csv

def test_read_csv_to_dicts():
    csv_content = """name,age,city
Alice,30,NY
Bob,25,LA
"""

    # Simulate a file with StringIO

```

```

fake_file = io.StringIO(csv_content)
reader = csv.DictReader(fake_file)
result = list(reader)
assert result == [
    {"name": "Alice", "age": "30", "city": "NY"},
    {"name": "Bob", "age": "25", "city": "LA"},
]

test_read_csv_to_dicts()
...

```

Running a tiny unit test like this catches the most common bugs (wrong delimiter, extra blank line, etc.) before the code lands in production.

6. TL;DR – the checklist you can hand to your mentee

1. ****Open**** with ``newline=""`` and the correct ``encoding``.
2. Use ****csv.DictReader**** – it reads the header for you.
3. ****Filter**** out completely empty rows if the source file may have them.
4. ****Collect**** into a list ``only`` if you need random access; otherwise return the iterator.
5. ****Don't**** try to convert types inside the CSV-reading function – keep it pure.
6. ****Validate**** with a small unit test or by printing the first few rows.

With those steps you'll have a clean, reliable way to turn any well-formed CSV into a list of dictionaries, ready for further processing. Happy coding!

```

=====
MODEL B (groq/compound-mini):
=====
### How to read a CSV file into a **list of dictionaries**

```

When you read a CSV with the built-in ``csv`` module, each row can be turned into a ``dict`` whose keys are the column headers and whose values are the cell contents.

The most straightforward way is to use ****csv.DictReader**** – it does the mapping for you automatically.

1. Core idea (step-by-step)

Step	What you do	Why it matters
1	**Open the file** <code>`open(path, newline="", encoding='utf-8')`</code> <code>`newline=""`</code> tells the CSV parser to handle line endings itself; explicit <code>`encoding`</code> avoids Unicode errors on non-ASCII data.	
2	**Create a `DictReader`** <code>`csv.DictReader(file_obj)`</code> It reads the first line as the header and returns each subsequent row as a <code>`dict`</code> .	
3	**Iterate & collect** <code>`list(reader)`</code> or a loop Turns the iterator into a concrete list (or lets you process rows one-by-one if you're worried about memory).	
4	**Optional Post-process** Convert strings → int/float, handle missing values, etc. CSV stores everything as text; you often need the correct Python types.	

2. Common pitfalls & how to avoid them

Pitfall	Symptom	Fix / Best practice
Forgot `newline`	Extra blank rows or <code>`\\r`</code> characters in fields.	Always open CSV with <code>`newline=""`</code> .
Wrong encoding	<code>`UnicodeDecodeError`</code> on characters like <code>`é`</code> or <code>`ø`</code> .	Use <code>`encoding='utf-8'`</code> (or the correct encoding for your file).
Header mismatch	Keys you expect aren't present, or you get <code>`None`</code> keys.	Verify the first line really contains the column names; if not, pass <code>`fieldnames=`</code> manually.
Trailing spaces in headers	Keys become <code>` name`</code> instead of <code>`name`</code> .	Use <code>`skipinitialspace=True`</code> or strip the header line yourself.
Large files	<code>`MemoryError`</code> when you do <code>`list(reader)`</code> on a huge CSV.	Process rows in a loop (<code>`for row in reader: ...`</code>) or use chunking/pandas if you need random access.
Missing values	Empty strings where you expect numbers.	Decide on a sentinel (<code>`None`</code> , <code>`0`</code> , etc.) and convert after reading.
Inconsistent quoting / delimiters	Rows shift columns, parsing errors.	Explicitly set <code>`delimiter=','`</code> , <code>`quotechar="'"`</code> , or use a custom <code>`dialect`</code> .
Mutable default values	Using <code>`row.get('col', [])`</code> and then mutating the list later can affect other rows.	Convert values per-row; never store mutable defaults in the dict itself.

3. Clean, production-ready example

```

```python
import csv
from pathlib import Path
from typing import List, Dict, Any

```

```

def read_csv_as_dicts(
 file_path: Path,
 *,
 encoding: str = "utf-8",
 delimiter: str = ",",
 skip_initial_space: bool = True,
 convert_types: bool = True,
) -> List[Dict[str, Any]]:
 """
 Read a CSV file and return a list of dictionaries.

 Parameters

 file_path: Path
 Path to the CSV file.
 encoding: str, default "utf-8"
 Text encoding of the file.
 delimiter: str, default ","
 Column separator.
 skip_initial_space: bool, default True
 Strip spaces after delimiters (helps with "a, b, c").
 convert_types: bool, default True
 If True, attempt to cast numeric strings to int/float and
 empty strings to None.

 Returns

 List[Dict[str, Any]]
 One dict per row, keys are column headers.
 """
 rows: List[Dict[str, Any]] = []

 # Open with newline="" so csv module handles line endings correctly
 with file_path.open(mode="r", newline="", encoding=encoding) as f:
 reader = csv.DictReader(
 f,
 delimiter=delimiter,
 skipinitialspace=skip_initial_space,
)

 for raw_row in reader:
 if not convert_types:
 rows.append(dict(raw_row)) # shallow copy, keep strings
 continue

 # Convert each value – this is a tiny helper that you can
 # replace with a more sophisticated schema if needed.
 clean_row: Dict[str, Any] = {}
 for key, value in raw_row.items():
 # Treat empty strings as missing data
 if value == "":
 clean_row[key] = None
 else:
 # Try int → float → keep as string
 try:
 clean_row[key] = int(value)
 except ValueError:
 try:
 clean_row[key] = float(value)
 except ValueError:
 clean_row[key] = value
 rows.append(clean_row)

 return rows

Example usage
if __name__ == "__main__":
 csv_path = Path("data/example.csv")
 data = read_csv_as_dicts(csv_path)

 # Quick sanity check
 for i, row in enumerate(data[:5], start=1):
 print(f"Row {i}: {row}")
 ...

```

### ### What the example shows

1. **Robust file opening** – `Path.open`` with ``newline=""`` and explicit ``encoding``.
2. **DictReader** – automatically maps header → value.
3. **Optional type conversion** – a tiny, reusable routine that turns `"42" → int``, `"3.14" → float``, `"" → None``.  
(In real projects you might use a schema library like ``pydantic`` or ``marshmallow``.)
4. **Typed signatures** – helps static analysis tools (``mypy``, VS Code) and documents intent.
5. **Safety for large files** – you could replace the ``rows.append`` with a generator (``yield clean_row``) if you need streaming.

---

### ## 4. Quick "one-liner" for small scripts

If you just need a quick read and you're sure the file fits in memory:

```
```python
import csv
from pathlib import Path

records = list(csv.DictReader(Path("data/example.csv").open(newline="", encoding="utf-8")))
```
```

**\*Remember:** the one-liner skips all the safety checks we added above, so use it only in throw-away scripts or notebooks.

---

### ### TL;DR for the mentee

- Use ``csv.DictReader`` – it does the heavy lifting.
- Always open the file with ``newline=""`` and the correct ``encoding``.
- Watch out for blank rows, stray spaces, and type conversion.
- If the CSV is huge, process rows in a loop instead of building a full list.

Feel free to ask if you'd like to see a version that streams rows (generator) or integrates with a validation library!

```
=====
MODEL C (openai/gpt-oss-20b):
=====
```

### ## TL;DR

```
```python
import csv

with open('data.csv', newline="", encoding='utf-8') as f:
    rows = [row for row in csv.DictReader(f)]
```
```

``rows`` is now a ``list`` where each element is a ``dict`` mapping column names to the cell value for that row.

---

### ## Why ``csv.DictReader``?

- Built-in** – no third-party libs needed.
- Handles the header row automatically.
- Keeps the original column names as keys, so you can access fields by name (``row['age']``).
- Works with any file-like object (file, ``StringIO``, etc.).

---

### ## Step-by-step

1. **Open the file**

```
```python
f = open('data.csv', 'r', newline="", encoding='utf-8')
```
```

  - \* ``newline=""`` tells the ``csv`` module to handle newlines itself; otherwise you may get blank lines on Windows.
  - \* Use ``with`` to ensure the file is closed automatically.
2. **Create a ``DictReader``**

```
```python
reader = csv.DictReader(f)
```
```

  - \* The first row of the file is read as the keys for all dictionaries.
  - \* If the file has no header, you can pass ``fieldnames=[...]'`` manually.
3. **Iterate and build a list**

```
```python
```

```
rows = list(reader)      # or [row for row in reader]
...

* Each 'row' is an 'OrderedDict' (Python 3.7+ it's a normal 'dict' with preserved order).
* If the file is huge, consider processing rows one-by-one instead of loading the whole file into memory.
```

4. **Use the data**

```
```python
for row in rows:
 print(row['name'], row['age'])
...

```

#### ## Common pitfalls & how to avoid them

Pitfall	Why it happens	Fix
**Blank lines in the output**	'newline=' missing.	Add 'newline=' in 'open()'.
**Wrong encoding**	File is UTF-8 but you open with default ASCII.	Specify 'encoding='utf-8' (or whatever the file uses).
**Missing header row**	File has no header but you rely on column names.	Pass 'fieldnames=...' to 'DictReader'.
**Large files causing memory errors**	Converting the whole iterator to a list loads everything.	Process rows lazily: 'for row in csv.DictReader(f): ...'
**Trailing whitespace in keys**	Header contains spaces like '" first name "'.	Strip keys: 'reader.fieldnames = [k.strip() for k in reader.fieldnames]'
**Mixed data types**	All values are strings.	Convert explicitly: 'int(row['age'])', 'datetime.strptime(row['date'], '%Y-%m-%d')', etc.

#### ## Clean, production-ready example

```
```python
import csv
from pathlib import Path
from typing import List, Dict, Iterable, Any

def read_csv_as_dicts(
    path: str | Path,
    *,
    encoding: str = 'utf-8',
    fieldnames: Iterable[str] | None = None,
) -> List[Dict[str, Any]]:
    """
    Read a CSV file into a list of dictionaries.

    Parameters
    -----
    path:
        Path to the CSV file.
    encoding:
        Text encoding of the file.
    fieldnames:
        Optional explicit field names if the CSV has no header row.
        If None, the first row is used as the header.

    Returns
    -----
    List[Dict[str, Any]]
        A list where each element represents a row.
    """
    with open(path, newline="", encoding=encoding) as f:
        reader = csv.DictReader(f, fieldnames=fieldnames)
        # Strip whitespace from keys – handy if the header contains accidental spaces
        reader.fieldnames = [name.strip() for name in reader.fieldnames]
        return list(reader)

# Usage
if __name__ == "__main__":
    data = read_csv_as_dicts('employees.csv')
    for row in data:
        print(f"{row['first_name']} {row['last_name']} → {row['department']}")
...
```
```

#### ### What this does

- \* \*\*Type hints\*\* make the intent clear.
- \* The helper function encapsulates the pattern so you can reuse it in tests or other modules.
- \* Explicit 'encoding' and optional 'fieldnames' give you flexibility for different CSV sources.
- \* Stripping whitespace from keys prevents subtle bugs when the CSV was produced by a human or a tool that added spaces.

---  
  
## Quick "cheat sheet"

```
```python
# 1. Open
with open('file.csv', newline="", encoding='utf-8') as f:
    # 2. Reader
    reader = csv.DictReader(f)
    # 3. List
    rows = list(reader)

# 4. Use
for row in rows:
    print(row['column_name'])
```
```

---  
  
### Final tip

If you find yourself doing a lot of data manipulation (type conversions, filtering, aggregation), consider moving to **pandas**:

```
```python
import pandas as pd
df = pd.read_csv('file.csv')
# df is a DataFrame; you can convert to dicts if needed:
rows = df.to_dict(orient='records')
```
```

Pandas is faster for large datasets and offers many utilities, but the pure-Python solution above is lightweight and perfect for most interview or teaching scenarios.

=====

MODEL D (qwen/qwen3.6-27b):

=====

Error querying the LLM: Error code: 429 - {'error': {'message': 'Request too large for model `qwen/qwen3.6-27b` in organization `org\_01jqcgtmc1e8ktt3wma9n5qf7q` service tier `on\_demand` on output tokens per minute (OTPM): Limit 1000, Requested 1979. The request's expected output tokens exceed the enforced limit; reduce max\_tokens (or the request's expected output) and try again. Need more tokens? Upgrade to Dev Tier today at <https://console.groq.com/settings/billing>', 'type': 'tokens', 'code': 'rate\_limit\_exceeded'}}

=====

EVALUATION (openai/gpt-oss-120b)

=====

- Which response is better: **\*\*A\*\***
- Brief explanation of why: Response A delivers a technically correct solution, explains every step clearly, and follows idiomatic Python practices (use of `Path`, explicit `newline=""`, type hints, docstring, generator expression, optional empty-line filtering). It also discusses common pitfalls, offers a minimal one-liner, and includes a tiny unit test, making it both educational and production-ready without over-complicating the code.
- Key strengths of the winning response:
  - **\*\*Correctness\*\*** Uses `csv.DictReader` properly; handles encoding, newline, and empty-line skipping.
  - **\*\*Clarity\*\*** Step-by-step walkthrough, tables summarizing actions and pitfalls, and concise "TL;DR" sections.
  - **\*\*Best practices\*\*** Pathlib for file paths, type annotations, docstring, generator for lazy filtering, advice on when to return a list vs. an iterator.
  - **\*\*Educational value\*\*** Highlights common mistakes, provides a quick one-liner, and includes a simple unit test for verification.

=====

What would you like to do next?

1. Continue conversation (ask another question, same technique)
  2. Start new conversation with a different prompting technique
  3. Exit
- Enter your choice (1, 2, or 3): 3

Goodbye!

himanshishrivats@Himanshis-MacBook-Air GenAI\_Content %

-----

# Reflection

- How different prompting techniques (zero-shot, few-shot, chain-of-thought, etc.) changed the responses for the same question.

**Ans.** Same question ("read CSV into list of dicts"), but each technique got longer and more structured as it went:

- **Zero-Shot** → short, just the code + one sentence.
- **Few-Shot** → matched the example format (small explanation + code block), stayed compact.
- **Chain-of-Thought** → broke it into steps, added a table, showed a full runnable example, even added output.
- **Role-Based** (mentor persona) → the longest and most "senior engineer" tone — added a pitfalls table, testing example, and a TL;DR checklist for a "junior."

**Simple takeaway:** the *question* didn't change, but the *instructions around it* changed how much the model explained, how it formatted things, and how "thorough vs. quick" it decided to be. That's the whole point of prompt engineering — you're not teaching the model new facts, you're steering its behavior.

- Whether certain models tend to be graded more favorably by the evaluator.

**Ans.** Yes, in the run, gpt-oss-120b (Model A) won 2 out of 3 rounds, and gpt-oss-120b was also the evaluator both times. Worth flagging in our code: the evaluator judging itself favorably is a known bias in LLM-as-judge setups - it's not proof A was objectively best, it's a real limitation we should name explicitly.

- How larger models perform versus smaller ones (e.g. `gpt-oss-120b` vs. `gpt-oss-20b`).

**Ans.** (120b Vs 20b) Both used `openai/gpt-oss-*`, so this is a clean comparison. From your output: 120b tended to give longer, more "polished" answers with tables and extra sections; 20b gave answers that were still fully correct but tighter and less padded. *Bigger didn't mean "more correct" here, both were right, it mainly meant more explanation and more format polish.*

- How different reasoning settings (`reasoning_effort`, `reasoning_format`, `include_reasoning`) affected the reasoning model's output.

**Ans.**

tested `reasoning_effort=medium` and `high` — both worked fine, and I didn't see much difference in output quality for this simple question. That makes sense since reasoning effort probably matters more on harder, multi-step problems, not something like reading a CSV file.

When tried `reasoning_format="parsed"` along with `include_reasoning`, three of my four models (A, C, D) failed with: `cannot specify both include_reasoning and reasoning_format`. This told me the Groq API treats these two params as mutually exclusive.

- What happens when you ask about recent events (e.g. "what is the latest version of Python?") — do all models handle this the same way

**Ans.** Got 3 different answers: A said 3.14.2, C said 3.12.6, and D said 3.12.x/3.13. B failed with a "Request Entity Too Large" error (likely because it inherited a long conversation history and the request grew too big).

This confirms none of these models can verify current facts, they're all just recalling whatever was true at their training cutoff, and those cutoffs clearly differ, since the models disagree with each other by multiple versions. None of them said "I'm not sure" or "I can't verify this in real-time," even though that would have been the most honest answer.

The evaluator picked D as best, but that's actually a weak call, D still gave outdated info, just phrased more cautiously with a hedge ("check the official website"). The evaluator rewarded *tone* (cautious phrasing) over *correctness*, which shows a real limitation of LLM-as-judge: it can't independently verify facts either, so it's not actually checking who's right, just who sounds most careful.

**Takeaway for my reflection:** this is a clean demonstration of hallucination risk on time-sensitive questions - confident, contradicting answers instead of admitting uncertainty. It also shows that an LLM evaluator can be fooled by hedged language into rating a still-wrong answer as "correct."